

Genesis Engine : un laboratoire déterministe et falsifiable pour l'émergence des civilisations

Auteur : Micka Delcato **Affiliation** : Chercheur indépendant — projet Genesis Engine
Correspondance : github.com/Micka420-collab/genesis-engine **Version** : Préprint v1 — 2026-05-28
Licence : AGPL-3.0-only (texte) ; CC-BY-4.0 (ce document) **Langues** : [English](#) · Français (ce fichier)

Résumé

Nous présentons **Genesis Engine**, un laboratoire de vie artificielle (*artificial life*, alife) open source dont l'objectif de conception est l'*observation de phénomènes émergents à l'échelle d'une civilisation* — langage, outils, commerce, monnaie et effondrement — sous une contrainte méthodologique stricte : **seules les lois de la physique sont codées en dur ; tout ce qui est culturel doit émerger des agents, et chaque affirmation d'émergence doit être falsifiable**. Le système couple un substrat Rust (un monde voxel déterministe, adressé par contenu, doté du climat de Köppen, de géologie et d'hydrologie) à une couche de simulation Python (génom, métabolisme, perception, neuro-évolution et dynamiques sociales). La reproductibilité est garantie par une source d'entropie unique — une fonction pseudo-aléatoire (PRF) — et par des *empreintes d'état* SHA-256 consignées dans des manifestes de provenance propres à chaque exécution. Nous formalisons quatre invariants au niveau du moteur (I-1...I-4) ainsi qu'un *registre de falsifiabilité* poppérien qui sépare les **observables vérifiés** des **hypothèses ouvertes**. Nous rapportons une démonstration empirique du déterminisme au niveau d'une exécution (graine identique → empreinte identique ; graine distincte → empreinte distincte), un observable génétique vérifié (le coefficient de consanguinité de Wright $F = 0,2500$ pour des germains) et un observateur épidémique déterministe. Nous sommes explicites sur ce qui n'est **pas** encore prouvé : monnaie endogène, effondrement à la Tainter, sélection sexuelle, bulles épistémiques et construction de niche bidirectionnelle demeurent des *hypothèses à tester*, et non des résultats. La contribution est moins un résultat unique qu'une **méthodologie** : un appareil dans lequel les affirmations sur l'émergence peuvent être préenregistrées, reproduites bit à bit, et réfutées.

Mots-clés : vie artificielle, émergence, déterminisme, reproductibilité, falsifiabilité, simulation multi-agents, évolution culturelle, systèmes complexes.

1. Vision et théorie

1.1 La thèse

La plupart des simulations de « civilisation » *scénarisent* le résultat : les règles contiennent déjà la réponse (arbres technologiques, recettes fixes, « le sel, c'est de la monnaie »). Genesis Engine adopte la position inverse, que nous appelons **ZÉRO PRÉ-SCRIPT** :

Si un phénomène est intéressant parce qu'il a émergé, alors il ne doit pas figurer dans les règles. Seule la physique est donnée. Le langage, les outils, la monnaie, la structure sociale et l'effondrement doivent surgir — ou échouer à surgir — de l'interaction d'agents incarnés avec un monde physique.

La filiation intellectuelle est la tradition de la vie artificielle (Tierra, Avida, Lenia, Polyworld) étendue à l'échelle *civilisationnelle*, et la philosophie des sciences de Karl Popper : une affirmation qui ne peut être réfutée n'est pas une affirmation scientifique. L'hypothèse de travail du projet n'est donc pas « la civilisation va émerger », mais l'énoncé plus fort et testable : « **sous la physique P, l'architecture d'agent A et la graine s, l'observable O franchit le seuil θ sur ≥ 3 graines indépendantes.** » Tout ce qui est plus faible est marqué comme question ouverte.

1.2 Pourquoi le déterminisme est porteur

Les affirmations d'émergence sont notoirement fragiles : un résultat qui apparaît une fois, sur une machine, sous un seul tirage aléatoire, est une anecdote, pas une preuve. Genesis Engine fait du déterminisme un *invariant de première classe*. Étant donné les mêmes (graine, configuration, région), le monde et sa trajectoire doivent être reproductibles bit à bit, et l'état résultant doit produire une empreinte SHA-256 identique. Cela transforme « j'ai vu la monnaie émerger » en « la monnaie a émergé dans l'exécution c2e03804... », reproductible par quiconque sur le même commit ». Le déterminisme est la colonne vertébrale à laquelle s'accroche la falsifiabilité.

1.3 Depuis l'origine de la vie

L'ambition à long terme est une pile qui commence au substrat (protocellules → photosynthèse → oxygénation → faune) et laisse des agents sages apparaître sans fondateurs scénarisés — « le libre arbitre comme origine », selon la formulation du projet lui-même. Ce document **ne prétend pas** que cet arc complet a été démontré. Il documente l'*appareil* et la *méthode*, et rapporte le sous-ensemble de comportements actuellement étayé par un test déterministe qui passe.

2. Contexte et travaux connexes

Tradition	Systèmes exemples	Ce que Genesis Engine emprunte	Ce qu'il change
Évolution numérique	Tierra, Avida	Auto-réplication, sélection sur un substrat	Ajoute un monde physique ancré sur la Terre et l'incarnation
Alife continue	Lenia, particle life	Émergence à partir de règles locales	Ajoute des agents discrets dotés de génomes et de cognition
Agents incarnés	Polyworld, agents neuronaux	Neuro-évolution (plasticité de type NEAT)	Couple au climat/géologie/hydrologie à l'échelle planétaire
Sciences sociales à base d'agents	Sugarscape, modèles NetLogo	Commerce, gradients de ressources	Interdit les institutions scénarisées ; elles doivent émerger
ML/science reproductible	Préenregistrement, <i>model cards</i>	Provenance + préenregistrement des hypothèses	Empreintes d'état SHA-256 par exécution, registre poppérien

L'engagement distinctif est méthodologique : non pas *que* des phénomènes émergent, mais que **chaque affirmation d'émergence porte sa propre condition de réfutation et une empreinte reproductible**.

3. Méthodologie

3.1 Source d'entropie unique (PRF)

Toute la stochasticité transite par une seule fonction pseudo-aléatoire, `engine.core.prf_rng`. L'usage de `random.*` non initialisé est interdit par les règles de contribution et par la revue. Cela garantit que `(graine, config)` détermine entièrement une exécution.

3.2 Invariants au niveau du moteur

Quatre invariants forment le contrat du substrat. Ils sont censés être gardés par des tests ; la §6.4 rapporte honnêtement l'écart d'application actuel pour trois d'entre eux.

ID	Invariant
I-1	Même <code>(graine, config)</code> → chunks identiques au bit près (mono- et multi-thread)
I-2	Hachage robuste aux NaN et multiplateforme dans chaque structure adressée par contenu
I-3	Coalescence concurrente : N appelants parallèles pour une coordonnée déclenchent exactement un <code>generate()</code> (robuste aux paniques)
I-4	Les chunks mutés survivent à l'éviction du cache et au snapshot/restore

3.3 Empreinte d'état et provenance

Chaque exécution longue est enveloppée dans `experimental_run`, qui écrit un `manifest.json` capturant : le commit git, le SHA-256 de `pyproject.toml`, la version de Python, la plateforme, le chronométrage, le `summary` de l'exécution, et une **empreinte d'état** = SHA-256 sur l'état de *simulation déterministe*. De façon cruciale, l'empreinte **exclut** les champs volatils liés à l'hôte/au temps (débit, temps mural, horodatages, chemins absolus) ; la §6.1 rapporte un défaut que nous avons trouvé et corrigé précisément dans ce domaine. Un manifeste est écrit même en cas de crash, de sorte que les exécutions partielles restent diagnosticables.

3.4 Préenregistrement et registre de falsifiabilité

Avant une exécution pilotée par une hypothèse, l'expérimentateur copie `PREREGISTRATION_TEMPLATE.md`, énonce l'hypothèse, la prédiction quantitative et la condition d'arrêt/réfutation, puis **le commit avant l'exécution** (le graphe git est la piste d'audit). Les résultats sont consignés dans `FALSIFIABILITY.md` avec l'un des cinq statuts : *confirmé* / *en attente* / *nul* / *réfuté* / *remplacé*. **La confirmation exige que l'observable franchisse son seuil sur ≥ 3 graines distinctes au même commit.** Aucune affirmation publique ne peut être citée sans son entrée au registre.

3.5 Discipline des Waves

Les fonctionnalités sont livrées sous forme de « Waves » numérotées, chacune accompagnée d'un script de fumée (`pnn_smoke.py`) et intégrée au tick principal. Cela maintient la surface des affirmations alignée sur la surface des tests : une Wave sans fumée verte n'est pas « terminée ».

4. Architecture du système

```
L4 Civilisation  commerce · construction · polité · langage · observateurs
L3 Cognition    perception locale · plasticité de type NEAT · sélection d'action
L2 Biologie     génome 256-D · métabolisme · anatomie · sélection sexuelle
L1 Monde        Genesis · climat de Köppen · biomes · ressources
L0 Physique     thermodynamique · gravité · hydrologie · érosion
```

▲
Substrat Rust (native/world-engine) : WorldGraph, streaming,
GPU, snapshot/restore – relié à Python via PyO3 (genesis_world)

- **Substrat Rust** (`native/world-engine/` , 24 crates) : un monde voxel adressé par contenu, découpé en chunks, à génération déterministe, avec classification de Köppen, géologie, hydrologie, météo, maillage et snapshot/restore. Exposé à Python via une roue (*wheel*) PyO3.
- **Runtime Python** (`runtime/engine/`) : les couches d'agents et de société — génome, métabolisme, perception, cognition, communication, construction, commerce, et une suite d'*observateurs* (épidémie, lignée, vision) qui mesurent plutôt qu'ils ne pilotent la simulation.
- **Pont** (`engine.rust_bridge`) : sélectionne le backend natif lorsqu'il est disponible et validé, sinon un mock Python ancré sur Genesis. La §6.2 rapporte un véritable défaut du pont que nous avons corrigé.

5. Reproductibilité et observables vérifiés

Ce qui suit est étayé par des tests qui passent de façon déterministe. Nous étiquetons chacun avec son gardien afin qu'un lecteur puisse le réexécuter.

5.1 Déterminisme au niveau d'une exécution (I-1 empirique)

En utilisant le pipeline du projet lui-même, nous avons exécuté un scénario de civilisation de 300 ticks à trois reprises :

```
python runtime/scripts/civilization_pipeline.py \
  --experiment paper --seed <S> --ticks 300 --founders 12
```

Exécution	Graine	Empreinte d'état (SHA-256, abrégée)
A	0xC1A71CE0	c2e038049950056e105503ff8430281617edab3a...
B	0xC1A71CE0	c2e038049950056e105503ff8430281617edab3a...
C	0xBADC0FFEE	7fe0b90867d29608960c4ea2abe939413701c4e4...

Résultat : A = B (graine identique → empreinte identique) et A ≠ C (graine distincte → empreinte distincte). La `world_signature` associée à la ligne de base est `7a6d7eb0bc1c8140205c772d2fc3935b66bd0ba9e4dac9647f997910b4b68304` . Provenance consignée : Python 3.14.3, Windows-10, SHA-256 de `pyproject.toml` `6c7e2555...` . Il s'agit d'une démonstration directe et reproductible de l'invariant de déterminisme au niveau du pipeline.

5.2 Structure génétique — coefficient de consanguinité de Wright

Gardien : `p71_lineage_observer_smoke` (9/9 RÉUSSIS). Pour des germains, l'observateur calcule $F = 0,2500$ exactement, et $F = 0,0000$ pour des paires non apparentées — les valeurs des manuels. Parce que l'observateur de lignée *mesure* le graphe du génome au lieu d'imposer un résultat, ceci est une preuve que le substrat d'hérédité est correct, et non qu'un résultat social a été scénarisé.

5.3 Dynamique épidémique — observateur déterministe

Gardien : `p70_epidemic_observer_smoke` (9/9 RÉUSSIS), y compris une vérification explicite du *déterminisme inter-exécutions* sur les instantanés du graphe de contacts. L'observateur suit un état de type SIR par pathogène sur un réseau de contacts émergent. Nous rapportons la machinerie et son déterminisme comme vérifiés ; les nombres spécifiques de bassin (p. ex. une valeur de R_0) dépendent du scénario de pathogène choisi et doivent être cités par scénario, et non comme une constante universelle.

5.4 Étendue du substrat (étayée par les fumées)

Le pipeline validé installe et fait avancer, de façon déterministe, les modules : `climate`, `genesis`, `geology`, `hydrology`, `marine`, `meteorology`, `wildfire`, plus un coupleur multi-cadence et la suite d'observateurs. Plus de 173 tests Python et une batterie de fumées `pNN` étayent le substrat ; voir les fichiers `NEXT-SPRINT.md` et `FALSIFIABILITY.md` du dépôt pour le décompte à jour.

6. Ce que nous avons corrigé pour rendre l'appareil scientifiquement valide

En préparant cet article, nous avons soumis l'appareil à des tests adverses et corrigé trois défauts qui touchent directement à la validité scientifique.

6.1 L'empreinte d'état n'était pas reproductible

L'empreinte d'état citable hachait la **totalité** de la charge utile de l'exécution, y compris le débit en temps mural (`tps`), les secondes en temps mural, et un `manifest_path` **absolu**. Deux exécutions de graine identique produisaient donc des empreintes *différentes* — réduisant silencieusement à néant la promesse centrale de reproductibilité du projet. Nous avons fait en sorte que `compute_state_fingerprint` supprime récursivement les clés volatiles liées à l'hôte/au temps avant le hachage, de sorte que l'empreinte ne reflète que l'état de simulation déterministe et soit stable **d'une machine à l'autre**. Le résultat $A = B$ de la §5.1 est la vérification post-correction ; des tests de non-régression verrouillent ce comportement.

6.2 Une collision de noms de roues pouvait casser silencieusement le pont natif

Deux crates Rust distincts compilaient vers un module Python du *même* nom (`genesis_world`) mais aux API *incompatibles*. Une roue héritée obsolète pouvait masquer la roue canonique dans `site-packages`, faisant planter le pont avec un `TypeError` cryptique. Nous avons durci le pont pour qu'il vérifie le contrat de l'API canonique `PyWorld` avant de faire confiance à un module comme « natif », se repliant sinon sur le mock ancré sur Genesis avec un avertissement exploitable. Cela transforme une corruption silencieuse en un état honnête et diagnosticable.

6.3 Rapport honnête du backend d'exécution

Les manifestes d'exécution consignent désormais `rust_bridge: {native: false, module: MockPyWorld}` lorsque la roue native est absente, plutôt que de surévaluer une exécution native. La validité scientifique exige que l'appareil rapporte *ce qu'il a réellement fait*.

6.4 Menaces sur la validité (ouvertes, divulguées)

- **Écart d'application des invariants.** Trois des quatre invariants du moteur (I-1, I-3, I-4) vivent dans des crates Rust dont l'étape de CI est actuellement marquée `continue-on-error`, c'est-à-dire que les échecs ne sont pas encore bloquants. L'affirmation publique selon laquelle les quatre sont « gardés par une CI bloquante » est donc plus forte que la réalité tant que ces crates ne compilent pas proprement et que le drapeau n'est pas retiré. (I-2 est véritablement bloquant.) Ceci est divulgué, non caché.
- **Empreintes mono-machine.** La §5.1 démontre le déterminisme sur une seule plateforme (Windows, Python 3.14.3). L'identité bit à bit multiplateforme est un *objectif* d'invariant (I-1/I-2) mais n'est pas démontrée ici.
- **Mock contre natif.** La §5.1 s'est exécutée contre le backend mock Python (la roue native était absente/héritée sur l'hôte de test). Le mock est ancré sur Genesis et déterministe, mais ce n'est pas le substrat Rust ; la reproduction étayée par le natif est un travail futur.
- **Le réalisme terrestre est partiel.** Le score de réalisme terrestre auto-évalué par le projet est d'environ 76 % (la géologie étant la dimension la plus faible). « Réaliste » est une direction, pas une affirmation achevée.
- **Version de Python.** Les résultats ont été produits sur Python 3.14, qui est *hors* de la plage de support déclarée ($\geq 3.11, < 3.14$) ; la suite passe néanmoins.

7. Hypothèses ouvertes (explicitement non prouvées)

Ce sont les phénomènes que l'appareil est conçu pour *tester*. Aucun ne peut être cité comme résultat tant qu'une exécution préenregistrée ne l'a pas validé sur ≥ 3 graines dans `FALSIFIABILITY.md`.

1. **Monnaie endogène.** Une marchandise devient un moyen d'échange uniquement à partir de transactions observées — sans « monnaie » codée en dur.
 2. **Effondrement à la Tainter.** Les rendements marginaux décroissants de la complexité conduisent les agents à refuser de construire/inventer et à revenir à la subsistance.
 3. **Sélection sexuelle.** La parade nuptiale couplée à une `mate_preference` dérivée de traits de personnalité produit une corrélation parent–descendant mesurable sur des traits visibles.
 4. **Bulles épistémiques.** La communication dotée d'un paramètre de véracité et de vérifications de cohérence produit des grappes de confiance émergentes ; les menteurs persistants deviennent isolés.
 5. **Construction de niche bidirectionnelle.** L'activité des agents (feu, agriculture, urbanisation) altère le substrat, qui à son tour exerce une pression sélective sur les agents.
-

8. Comment reproduire

```
git clone https://github.com/Micka420-collab/genesis-engine.git
cd genesis-engine
python -m venv .venv && . .venv/bin/activate      # Windows : .venv\Scripts\activate
python -m pip install -e ".[dev]"
make test-python      # suite de tests Python
# Reproduire le déterminisme de la §5.1 :
PYTHONPATH=runtime python runtime/scripts/civilization_pipeline.py \
    --experiment repro --seed 0xC1A71CE0 --ticks 300 --founders 12
# → runtime/experiments/repro_<UTC>/manifest.json (comparer state_fingerprint)
```

Pour enregistrer une nouvelle hypothèse, copiez `runtime/experiments/PREREGISTRATION_TEMPLATE.md`, remplissez-le, committez-le **avant** l'exécution, puis consignez le résultat dans `FALSIFIABILITY.md`.

9. Conclusion et appel à collaboration

La contribution de Genesis Engine est un *appareil épistémique* : un monde déterministe, ancré sur la Terre, dans lequel l'émergence à l'échelle d'une civilisation peut être préenregistrée, reproduite bit à bit, et réfutée. Nous avons démontré le déterminisme au niveau d'une exécution, un substrat génétique correct et un observateur épidémique déterministe ; nous avons été explicites sur le fait que les phénomènes phares (monnaie, effondrement, sélection sexuelle, bulles épistémiques, construction de niche) demeurent des **hypothèses**, et nous avons divulgué les limitations actuelles de l'appareil plutôt que de les masquer. Nous invitons les chercheurs en alife, les ingénieurs Rust/Python, les géographes et les philosophes des sciences à préenregistrer des hypothèses, à tenter des réfutations, et à reproduire ou casser les empreintes rapportées ici. *Une affirmation qui ne peut être réfutée n'est pas un résultat — alors venez essayer de réfuter celles-ci.*

Références (sélection, indicative)

1. K. R. Popper, *The Logic of Scientific Discovery*, 1959.
2. T. S. Ray, « An approach to the synthesis of life » (Tierra), *Artificial Life II*, 1991.
3. C. Ofria & C. O. Wilke, « Avida: A software platform for research in computational evolutionary biology », *Artificial Life*, 2004.
4. B. W. Chan, « Lenia — Biology of Artificial Life », *Complex Systems*, 2019.
5. L. Yaeger, « Computational genetics, physiology, metabolism... (PolyWorld) », *Artificial Life III*, 1994.
6. K. O. Stanley & R. Miikkulainen, « Evolving Neural Networks through Augmenting Topologies (NEAT) », *Evolutionary Computation*, 2002.
7. J. H. Epstein & R. Axtell, *Growing Artificial Societies* (Sugarscape), 1996.
8. J. A. Tainter, *The Collapse of Complex Societies*, 1988.
9. S. Wright, « Coefficients of inbreeding and relationship », *The American Naturalist*, 1922.
10. W. Köppen, « Das geographische System der Klimate », 1936.

Ce document fait partie du dépôt Genesis Engine et est versionné aux côtés du code qu'il décrit. Les empreintes et les décomptes sont valides à la date du commit référencé dans le dépôt au moment de la publication ; réexécutez les commandes de la §8 pour vérifier par rapport à l'arbre courant.